

0、初始准备

本地部署好相应的模型，本项目默认采用模型为 `pangu_embedded_7b`，默认嵌入模型为 `nvidia/NV-Embed-v2`。

1、配置环境

创建并切换到对应 `conda` 环境 执行命令：

```
pip install -r requirements.txt
```

2. 测评 LoCoMo 数据集

2.1 任务执行

使用以下命令启动后台测评任务。执行前请确保 `llm_base_url` 的网络连通性。

```
nohup python batch_run_locomo_outputs.py \  
  --llm_name "your_llm_name" \  
  --llm_base_url "llm_base_url" \  
  --embedding_name "your_embedding_model_name" \  
  --base_save_dir "save_dir" \  
> run_locomo.log 2>&1 &
```

2.2 产出物结构说明

任务运行产生的中间文件、缓存以及最终结果将存放在 `save_dir` 目录下：

```
save_dir/                                     # base_save_dir  
├── locomo_0/                                # 数据集子文件夹 (0-9)  
│   ├── llm_cache/                          # LLM 查询缓存  
│   ├── pangu_embedded_.../                 # 具体模型权重对应的输出  
│   │   ├── chunk_embeddings/               # 分块向量  
│   │   ├── entity_embeddings/              # 实体向量  
│   │   ├── episodic_memories/              # 情景记忆存储  
│   │   ├── fact_embeddings/                # 事实向量  
│   │   ├── fact_metadata.json              # 事实元数据  
│   │   └── graph.pickle                    # 图结构数据  
│   ├── openie_results_...json              # 开放实体抽取结果  
│   └── results.json                        <-- 核心产出：待评测的回答文件  
├── locomo_1/  
├── locomo_2/  
└── ...
```

2.3 评测与指标提取

获取 `results.json` 后，需执行评测脚本以计算指标：

1. 修改配置

编辑 `eval/eval_locomo.sh` 文件：

- 指向 `save_dir` 中的结果文件路径。
- 设置评测结果的保存名称。

2. 执行评测

```
bash eval/eval_locomo.sh
```

3. 查看结果

- **评测统计**：输出结果存放在 `results/` 文件夹下。
- **明细文件**：见 `out_file` 指定的提取文件。

3、测评longMemEval数据集

3.1 任务执行

使用以下命令启动后台测评任务。执行前请确保 `llm_base_url` 的网络连通性。

```
nohup python batch_run_longmemeval_outputs.py \  
  --llm_name "your_llm_name" \  
  --llm_base_url "llm_base_url" \  
  --embedding_name "your_embedding_model_name" \  
  --base_save_dir "save_dir" \  
> run_longmemeval.log 2>&1 &
```

3.2 产出物结构说明

任务运行产生的中间文件、缓存以及最终结果将存放在 `save_dir` 目录下，目录结构参考[2.2 产出物结构说明](#)。其中的 `locomo_i` 变为 `longmemeval_s_i` (*i* 从0到499) 输出结果存放在 `save_dir` 文件夹下对应目录的 `results.json` 文件下

3.3 评测与指标提取

日志记录在 `log_episodic` 文件夹下对应目录

1. 修改配置

编辑 `eval/eval_longMemEval.sh` 文件：

- 指向 `save_dir` 中的结果文件路径。
- 设置评测结果的保存名称。

2. 执行评测

```
bash eval/eval_longMemEval.sh
```

3. 查看结果

- 评测统计：输出结果存放在 `results/` 文件夹下。
- 明细文件：见 `out_file` 指定的提取文件。

4、当前实验结果

4.1 locomo数据集

评测结果对比

下表展示了不同方法在 LoCoMo 数据集上的表现

方法	Avg F1	Avg EM	LLM Accuracy
Context (仅相关上下文)	0.386	0.114	0.6928
向量检索	0.461	0.291	0.5156
hippoRAG2	0.446	0.279	0.5096
episode (ours)	0.486	0.308	0.5611
提升幅度	↑ 5.42%	↑ 5.84%	↑ 8.82%

说明：

1. 指标基准：**Context (仅相关上下文)** 为理论最大值（即假设检索完全准确，直接将相关上下文输入模型）。
2. 计算方式：指标提升率基于 $(\text{ours} - \text{max_baseline}) / \text{max_baseline}$ 。

4.2 longMemEval数据集

评测结果对比

下表展示了不同方法在 longMemEval 数据集上的表现

方法	Avg F1	Avg EM	LLM Accuracy
Context (仅相关上下文)	0.2260	0.0320	0.6860
向量检索	0.1653	0.0200	0.5720

方法	Avg F1	Avg EM	LLM Accuracy
hippoRAG2	0.3482	0.2510	0.4659
episode (ours)	0.4550	0.2960	0.6080
提升幅度	↑ 30.67%	↑ 17.93%	↑ 6.29%

💡 说明：

1. 指标基准：Context (仅相关上下文) 为理论最大值 (即假设检索完全准确，直接将相关上下文输入模型)。
2. 计算方式：指标提升率基于 $(\text{ours} - \text{max_baseline}) / \text{max_baseline}$ 。
3. 指标说明：Context与向量检索的回答由官方脚本生成，由于开启了 **CoT (Chain of Thought)** 以追求更高的推理准确率，导致其F1/EM 偏低。

5. 致谢与声明

本项目使用openPangu，遵循openPangu Model License Agreement Version1.0的条款和条件，旨在允许合理使用并促进人工智能技术的进一步发展。详情请参阅仓库目录下的LICENSE文件。